Contents

Optimizing a linear fractional function over an integer efficient set **1**
The problem . . . 1
One iteration . . . 1
Usage . . . 2
Larger instances . . . 2
How far it goes . . . 3
The fractional generalisation (MOILFP) . . . 4
Anytime: a certified gap . . . 5
Three independent references . . . 6
Design notes . . . 6
A second method: searching criterion space . . . 8
An exact–metaheuristic hybrid . . . 10
The augmented weighted Tchebychev program: a reference from the literature . . . 11
What an explicit `E(P_D)` buys . . . 12
How far the method reaches . . . 13
Modules . . . 13

Optimizing a linear fractional function over an integer efficient set

Pure-Python, dependency-free, exact-arithmetic implementation of

> Leila Younsi-Abbaci, Optimizing a linear fractional function over an integer efficient set, **Reliability: Theory & Applications**, No 4 (40), Vol. 11, March 2025.

The problem

Given the multi-objective integer linear program

```
(P_D)   "max"  Z_i = C_i x ,  i = 1..p
        s.t.   x in D = { x in Z^n_+ : A x <~ b }
```

and a decision maker's linear fractional criterion, solve

```
(P_E)   max  Phi(x) = (U'x + alpha) / (V'x + beta)
        s.t. x in E(P_D)                    <- the *efficient* points of (P_D)
```

`E(P_D)` is a union of faces of `D`: non-convex, with no explicit description. `(P_E)` is therefore a global optimisation problem with, in general, many local optima. The method reaches the global optimum **without enumerating `E(P_D)`**.

One iteration

Step	What it does	Where it lives
1	`P^l_RF`: maximise `Phi` over the truncated region `D_l` → an **upper bound** on `Phi` over every remaining efficient point	`milp.solve_fractional_milp`
2	Efficiency test of the optimum `x_l` (Theorem 1, Isermann). `psi* = 0` ⇔ efficient; otherwise the test returns an efficient point `x~_l` dominating it	`efficiency.test_efficiency`

Step	What it does	Where it lives
3	`Q(x~_l)`: best `Phi` among the points sharing the non-dominated vector `C x~_l` — all efficient, and all about to be cut away	`efficiency.best_with_same_criterion`
4	Walk the edges of `Gamma_l = { j : gamma_j = 0 }` (alternative optima, Definition 2). An efficient point there attains the upper bound ⇒ stop	`edges.explore_edges`
5	Sylva–Crema cut: delete `{ x : C x <= C x~_l }` and loop	`efficiency.add_sylva_crema_cut`

Termination (Proposition 3): each iteration removes at least one non-dominated criterion vector, and that set is finite for a bounded integer program.

Usage

```
from lfp_efficient import Model, MOILP, FractionalObjective, LE
from lfp_efficient import optimize_over_efficient_set

D = (Model(2)
     .add([-2, 1], LE, 0)
     .add([6, 1], LE, 21)
     .add([-2, 4], LE, 6))

problem = MOILP(D, criteria=[[1, -2], [-1, 4]])          # max Z1, max Z2
phi = FractionalObjective(U=[1, 1], V=[5, 1], alpha=-1, beta=-1)

solution = optimize_over_efficient_set(problem, phi, verbose=True)
print(solution.x, solution.value)        # [3, 3]  5/17
```

```
python3 examples/paper_example.py        # reproduces section 4 of the paper
python3 examples/large_example.py        # larger instances, up to |D| = 34635
python3 examples/scaling.py              # the scaling study, up to n = 20
python3 examples/fractional_example.py   # MOILFP: every criterion a ratio
python3 tests/test_lfp_efficient.py      # 112 tests, incl. the fractional case
```

Larger instances

The paper's illustration has 11 feasible points, small enough that any method works. `examples/large_example.py` runs the algorithm on instances up to `n = 10` variables and `p = 3` criteria, every answer cross-checked against an independent exact reference:

instance	|D|	Phi_opt	iter	generated	algorithm	check
medium n=4	106	15/7	5	5 of 31 efficient (16%)	0.41 s	0.01 s
medium n=5	250	15/7	5	5 of 54 efficient (9%)	0.33 s	0.03 s

instance	|D|	Phi_opt	iter	generated	algorithm	check
medium n=6	629	2	8	8 of 71 efficient (11%)	1.46 s	0.07 s
large n=10	31833	43/11	1	1	0.01 s	6.42 s
hard n=10	4994	30/11	3	3	0.38 s	1.28 s
hardest n=10	34635	13/23	11	11	26.8 s	7.62 s

The three `n = 10` rows show the spread in difficulty. In the large one the maximiser of `Phi` over `D` happens to be efficient, so the first efficiency test settles the problem — 31833 feasible points solved in 0.01 s, six hundred times faster than merely scanning them. In the hard and hardest ones the criteria reward large `x` while `Phi` rewards small `x`, so the maximiser of `Phi` is dominated and the algorithm has to cut its way through several non-dominated vectors. That is the regime the method is written for, and the one where the cost sits: each Sylva–Crema cut adds `p` binaries and `p+1` rows, so the sub-problems grow with the iteration count.

The middle rows are where the paper's claim is visible: the optimum is reached after generating 9–16 % of `E(P_D)`.

How far it goes

`examples/scaling.py` walks the same family up to `n = 20`:

Every answer is **proved optimal**, none of them by enumerating the region:

instance	Phi_opt	iterations	solve	certificate
n=10 ub=3	29/19	4	0.76 s	proved, 1.4 s (3288 challengers, 3 tests)
n=12 ub=3	5/17	9	15.7 s	proved, 7.8 s (16158 challengers, 7 tests)
n=12 ub=3	3/23	5	3.2 s	proved, 17.6 s (46366 challengers, 4 tests)
n=14 ub=3	4/37	4	6.9 s	proved, 164 s (362293 challengers, 3 tests)
n=16 ub=3	7/8	5	7.3 s	proved, 2.6 s (3064 challengers, 1 test)
n=20 ub=2	1/6	6	9.6 s	proved, 21.4 s (22905 challengers, 6 tests)

What a proof costs depends on where the optimum sits, not on `n`: the `n = 16` row is settled in 2.6 s because `Phi_opt = 7/8` leaves only 3064 points above it, while `n = 14` needs 164 s for 362293 challengers — and still only 3 efficiency tests, because the dominance witnesses already in hand absorb the rest.

`n` is not what decides the cost. The same `n = 16` is solved in 7 seconds on a tight feasible region and is still running after a minute on a loose one — while `n = 25` is proved optimal in 5.5 s and `n = 30` in 16.5 s, both in three cut iterations. What drives the cost is the number of cut iterations — one per non-dominated vector generated, each adding `p` binaries and `p+1` rows to every later sub-problem — and how hard `max Phi` over the truncated region is as an integer program.

Worth noting where the difficulty is **not**: `max Phi` over `D` with no cut yet is settled in a single branch & bound node at every size tested, `n = 30` included. The whole cost sits in the late iterations, where the disjunction "improve at least one criterion beyond every vector found so far" is what makes the sub-problem combinatorial. Sharpening the cut's big-M constants against the current region was tried there and measured: the bound improved too little to pay for its own linear programs (48.7 s → 50.5 s on the `n = 10` suite), so it is not in the code.

The fractional generalisation (MOILFP)

The paper's criteria are linear. The package solves the general case too, where **every criterion is a ratio**:

```
(MOILFP)  "max"  Z_k(x) = (c_k'x + a_k) / (d_k'x + b_k),  k = 1..p
          s.t.   x in D
```

A linear criterion is the degenerate case `d_k = 0`, `b_k = 1`, so `MOILP` is a thin constructor over `MOILFP` and the linear case is not a separate code path — it is the same code with unit denominators. The paper's 20 tests pass unchanged against the generalised code, which is the regression proof of that.

Everything is expressed once, through the linear form

```
e_k(x ; a) = (D_k(a) c_k - N_k(a) d_k)'x + (D_k(a) a_k - N_k(a) b_k)
           = D_k(a) * D_k(x) * ( Z_k(x) - Z_k(a) )
```

which is **linear in `x`** yet carries the **sign** of `Z_k(x) - Z_k(a)`, both denominators being positive. With integer data it is integer-valued on integer points, so the three questions the method keeps asking are exact integer linear conditions — `e_k >= 1` strictly better, `e_k == 0` equal, `e_k >= 0` at least as good. The `+1` of the linear cut transposes to ratios with **no minimal step to estimate**. On a linear criterion `e_k` collapses to `C_k x - C_k a`.

One thing genuinely changes. With linear criteria the maximiser of the efficiency test is itself efficient (Ecker & Kouada), because the test's objective is then exactly `sum_k (Z_k(x) - Z_k(a))`, monotone in `Z`. With ratios the k-th term carries a factor `D_k(x)` that varies from point to point, so the maximiser only **dominates** — a dominated point can outscore an efficient one. The dominance chain has to be walked (`repair_to_efficient`).

This is measured, not assumed. Over random instances:

criteria	dominance chains from a non-efficient point
linear	180 chains, **all of length 1**
fractional	**7 of 172 longer than 1**

Reusing the linear result on ratios would therefore be silently wrong on a measurable share of points, and `tests/` fails if a sample ever shows otherwise.

Validation of the fractional case: the exact test is checked against Definition 1 on **every feasible point** of random instances (234 points, all classified correctly), and the whole algorithm against brute force on 15 fully fractional instances.

```
python3 examples/fractional_example.py
```

(The *e_k* form, and the observation that the same theorem covers both cut regimes, are due to the *claude/lnatawruh-8gyw92* branch, which develops MOILFP on a separate numpy/scipy base.)

Anytime: a certified gap

Step 1 computes the maximum of `Phi` over the truncated region and the loop uses it only to decide whether to stop — so a valid **upper bound on the answer is produced every round and thrown away**. It is valid because after cutting on `x^1..x^l`, every efficient point either lies in a removed set (where it is dominated, or on a slice whose best `Phi` the `Q` sub-problem already folded into `Phi_opt`) or in the region step 1 searches:

```
max_E Phi  <=  max( Phi_opt, max{ Phi(x) : x in D_l } )
```

Reporting it makes the method **anytime**. `time_budget` stops the run and returns a real solution (attained at a known efficient point), a bound, and the distance between them:

```
sol = optimize_over_efficient_set(problem, phi, time_budget=10)
sol.value          # lower bound, attained at an efficient point
sol.upper_bound    # certified upper bound
sol.gap            # absolute remaining uncertainty
sol.gap_closed     # fraction of the starting gap eliminated
sol.proved_optimal # True when the two meet
```

The budget also reaches **inside** step 1: an interrupted branch & bound still returns the largest bound left open in its tree, which no feasible point can exceed, so the answer stays certified even when no sub-problem finished. Two details matter and are enforced by tests:

- **The running minimum, not the latest bound.** A completed step 1 returns the exact maximum over the region; an interrupted one returns a relaxation value that can sit above the exact maximum of a larger, earlier region. Keeping the tightest bound seen is what stops a longer run reporting a worse gap than a shorter one.
- **Absolute gap, never relative to the incumbent.** `(UB - value)/|value|` is a mixed-integer-programming habit that assumes objectives bounded away from zero. `Phi` is a ratio that can be negative: a real gap of 3.56 against an incumbent of −0.099 prints as “3599%” and reads as a broken method. Scale by the starting gap instead.

instance	2 s	10 s	30 s
`n=16` loose	69% closed	72% closed	77% closed
`n=20` loose	54% closed	70% closed	73% closed
`n=25`	86% closed	**proved**	proved
`n=30`	88% closed	88% closed	**proved**

The shape is the one that justifies an anytime method: most of the uncertainty goes in the first seconds, the last tenth is expensive or never arrives. The reason is structural — the bound is a maximum over a region that still contains non-efficient points, and those sit far above the efficient optimum (on the instances above, 6× to 15×), so it has to grind that band away one cut at a time. A bound that excludes non-efficient points without enumerating them would be the real improvement.

Credit: the observation that this bound was already being computed and discarded comes from the *claude/verify-correctness-wzqzp0* branch, which established it on a separate Gurobi-based implementation; it is re-derived and re-validated here against *lfp_efficient*.

Three independent references

Trusting a single implementation to check itself proves nothing, so the package carries three reference methods that share no code path with the algorithm:

function	how it works	cost
`enumerate_efficient_set`	enumerate the box, filter by Definition 1	`O(|D|^2)` pairwise dominance
`maximize_by_full_enumeration`	generate every non-dominated vector by repeated cuts, maximise `Phi` on each slice	this is the naive method the paper avoids
`best_over_efficient_set_by_scan`	sort `D` by decreasing `Phi`, return the first point surviving a dominance test	a handful of tests in practice
`certify_optimum`	prove the answer is efficient **and** that every feasible point with a greater `Phi` is dominated	never builds `D` or `E(P_D)`

The third one is the practical verifier: the first efficient point in `Phi`-decreasing order is the optimum of `(P_E)`, so the efficient set never has to be built. On the paper's example it answers after 3 dominance tests; on the 31833-point instance, after 1.

The fourth is what remains once even `D` is too large to enumerate. An answer to `(P_E)` is correct exactly when the returned point is efficient and no feasible point with a strictly greater `Phi` is admissible. Both are checked directly. The second part only concerns `{ x in D : Phi(x) > value }`, and with a positive denominator that set is carved out by **one extra linear row** (`Phi(x) > v` is `(U - vV)'x + (alpha - v*beta) > 0`), which the search prunes on like any other constraint. Almost every challenger is then discarded by a dominance witness already in hand, so tens of thousands of them cost a handful of exact efficiency tests: 46366 challengers settled with 4 tests on one `n = 12` instance.

Design notes

Exact rational arithmetic everywhere. The algorithm rests on exact predicates — is *psi** zero?, is *gamma_j* zero?, is this component an integer? With floats each of those needs a tolerance and silently misclassifies degenerate vertices. `fractions.Fraction` costs nothing at these sizes.

A hand-written tableau simplex instead of a solver. The method is written in the language of the simplex tableau: it needs the basis `B_k`, the updated columns `y_{k,j} = B_k^{-1} a_{k,j}` and the reduced gradient `gamma_k`. A black-box MILP solver returns none of that. Linear and linear fractional objectives share the pivoting machinery and differ only in their pricing rule (`LinearPricing` / `FractionalPricing`, the latter being Cambini–Martein's reduced gradient, i.e. Theorem 3 of the paper).

Denominators that change sign. Linear fractional programming assumes `V'x + beta > 0` on the feasible set — that is what makes `Phi` pseudo-concave and the branch-and-bound bound valid. The paper's own example violates it: `x = (0,0)` is in `D` and gives `5·0 + 0 − 1 = −1 < 0`; it is exactly where the paper reads `Phi_sup = Phi(0,0) = 1`. `solve_fractional_milp` therefore splits the feasible set along the sign of the denominator and solves both branches (on the negative one, `Phi = (−U'x−alpha)/(−V'x−beta)` is a proper LFP), taking the better. Points where the denominator vanishes are excluded — `Phi` is undefined there.

Deviations from the printed pseudo-code. "Algorithm 2: part 2" is internally inconsistent — it stores `X_opt = x_l`, the point just shown not to be efficient, while setting `Phi_opt = Phi(x~_l)`,

and re-solves P_l inside the branch that has just solved it. The implementation keeps the mathematics (bound / test / Q / cut / edges) and fixes the bookkeeping: the stored point is the efficient one that realises Phi_opt. One sound early stop is added — if the upper bound of step 1 does not beat the incumbent, no remaining efficient point can, so the search ends there. The edge walk only ever accelerates: every candidate is re-validated against D and re-tested for efficiency before use, and the early stop it triggers is conditioned on the value actually matching the upper bound.

The edge step of Definition 2, and what it is actually worth. Step 4 walks the edges E_j of Gamma_l = { j in N_l : gamma_j = 0 } looking for an efficient integer point that attains the round's upper bound. Two things had to be repaired before it did anything at all, and the measurement afterwards is still sobering.

It was reading the wrong tableau. The basis of Definition 2 belongs to the truncated region D_l; branch and bound leaves behind the tableau of the node that produced x_l, carrying that node's bound rows. Those rows pin variables, so their slacks sit basic at zero and the ratio test collapses — measured, 93 % of the edges had theta0 = 0 and never started. clean_tableau_at now rebuilds a basis of D_l at x_l from scratch (simplex.tableau_at), with no bound rows in it. It returns None when x_l is not a vertex of D_l — an integer optimum need not be one, and that happens on 38 of 170 rounds.

The step was measured in the wrong region. theta0 read off the tableau is bounded by every row of D_l, cut rows included, and those stop the walk long before x itself would leave D: 149 of the 154 edges with any room at all had a minimum ratio below 1, so the integer step floored to zero. max_step_in bounds the walk by D instead. Nothing is lost by that: the step's claim is that an efficient point scores the upper bound, which rests on gamma_j = 0 holding Phi constant along the whole edge, on the point being validated against D, and on its efficiency being tested — never on it satisfying the current cuts.

What it is worth. Of 174 zero-gradient columns over 80 random instances, only **8** move x at all: once cuts accumulate, most of Gamma_l is the cut machinery itself — a column that leaves every model variable fixed has rU_j = rV_j = 0 and so gamma_j = 0 automatically. Those are now skipped outright (edge_direction returns the zero vector), which is where the step's cost went. It then fires on about **one instance in eighty**, and saves one iteration when it does. tests/ pins the case that fires.

On run time it is close to free and close to worthless. Interleaved, three runs each, repaired code against the code before it:

instance	before	after
medium n=6	1.38 s	1.25 s
hard n=10	0.38 s	0.36 s
hardest n=10	23.83 s	24.04 s

Same optimum and same iteration count everywhere. The only gain outside the run-to-run spread is the small one on medium n=6; on the instance that actually costs something the two are indistinguishable. So the repair is worth having because the step is correct now rather than decorative — not because it makes the method faster.

A limit left standing. At a degenerate vertex one basis exposes only some of the incident edges, and the completion rule here (positive variables first, then slacks by descending index) picks one arbitrarily. Enumerating the bases of a degenerate vertex would expose the rest; given the payoff measured above, it is not worth the work.

Batch cutting (batch_cuts_after), and why it is off by default. The run time is (number of step 1 solves) × (size of the region), and the region grows by p binaries and p+1 rows per cut. Two ways to attack that, one of which does not work:

Handing the method a good incumbent does not reduce the iteration count. Measured by seeding `Phi_opt` with the **true optimum** at iteration 1:

instance	as it is	seeded with `Phi*`
`medium n=6`	8 iters, 1.23 s	**8** iters, 1.20 s
`hard n=10`	3 iters, 0.35 s	**3** iters, 0.27 s
`hardest n=10`	11 iters, 23.5 s	**11** iters, 14.4 s

Not one iteration saved anywhere. What gets cut is decided by the efficiency test on `x_l`, not by the incumbent, so the cut sequence is identical either way. An incumbent buys one thing only — a cutoff that prunes the branch & bound inside step 1, worth 39 % on the heavy instance and nothing on the others. That 39 % is a **ceiling**: a real heuristic returns a value `<= Phi*`.

Batching does reduce it. For `w > 0` a maximiser of `w'Z` over `D` is efficient, and with linear criteria that is one ordinary integer program — no cut, no binary, no efficiency test. So once the loop has shown it will be long, generate `p+1` such points, bank each one's `Q`, and cut on all of them at once:

instance	off	`batch_cuts_after=3`
`hardest n=10`	11 solves, 27.1 s	**8** solves, **15.7 s**
`rand n=7 s3`	6 solves, 1.42 s	**5** solves, **1.07 s**
`medium n=6`	8 solves, 1.45 s	8 solves, **2.20 s**

Over 21 instances the total falls 34.3 s → 23.6 s — and **essentially all of it is the one heavy instance**. Elsewhere the extra cuts are a bet that those points are ones the loop would have had to cut anyway, and when the bet loses the model has grown for nothing: on `medium n=6` the solve count does not move at all and the run is half again as long.

Two refinements were tried and reported rather than buried. Filtering out generated centres already inside an existing cut changes **nothing** (23.70 s → 23.59 s): they are genuinely new non-dominated vectors, just not ones on the path — which no filter can know in advance. Raising the trigger to 5 removes the tax on the cheap instances and gives most of the win back (23.6 s → 26.8 s).

So it is insurance for the heavy tail, not a general speed-up, and it is off unless asked for. It needs linear criteria and says so rather than silently doing nothing: `w'Z` is a sum of ratios in the fractional case, not a linear objective.

A second method: searching criterion space

`criterion_space.optimize_in_criterion_space` answers the same question and agrees with the method above everywhere. It differs in **where it keeps track of what is left to search**, and that turns out to decide the cost.

One Sylva–Crema cut says: delete `{ x : Z(x) <= Z(x~) }`. In criterion space that is one box subtraction. Written in **decision** space it is a disjunction — some criterion strictly improves — and a disjunction needs `p` binaries and `p+1` big-M rows. That is the entire source of the model growth: eleven cuts on the heaviest instance leave 33 binaries and 44 rows on top of `D`, which is why the late sub-problems cost so much more than the early ones.

Keep the remainder as a **list of boxes** instead, and every sub-problem is `D` plus a handful of ordinary linear rows — no binary anywhere, and **the model never grows**.

instance	decision space	criterion space	
`hardest n=10`	11 iters, 23.47 s	49 boxes, **2.61 s**	**9.01×**
`n=8 s5`	7 iters, 2.97 s	19 boxes, **0.45 s**	6.56×
`n=8 s3`	4 iters, 0.93 s	16 boxes, **0.20 s**	4.66×
`medium n=6`	8 iters, 1.24 s	46 boxes, **0.52 s**	2.38×
`hard n=10`	3 iters, **0.36 s**	13 boxes, 0.41 s	0.86×

Over 35 instances: **40.65 s → 8.06 s, 5.05×, faster on 34 of them**, the same optimum everywhere and both proved optimal. The margin widens with difficulty — which is the signature of a structural change rather than a lucky instance. The one regression is the case the decision-space method settles in three iterations, where paying 13 boxes is a net loss.

How a box is written, and why not with numbers. Not as numeric bounds on `Z_k`: those are rationals in the fractional case and the exact `+1` the split needs would be lost. A box is a list of rows in the linear form of Theorem 4, `e_k(x ; a) = D_k(a) · D_k(x) · (Z_k(x) − Z_k(a))`, which is linear in `x` and **integer-valued**, so `Z_k(x) > Z_k(a)` is exactly `e_k(x) >= 1`. Linear and fractional criteria therefore go through the same code, and MOILFP is supported from the start rather than bolted on.

The split. Removing `{ Z <= Z(a) }` leaves `p` **disjoint** children: child `k` takes `e_k(· ; a) >= 1` together with `e_j(· ; a) <= 0` for every `j < k`. A point outside the removed set has a smallest index where it beats `Z(a)`, and that index picks its child. `tests/` checks this directly rather than through the answer: for every centre and every feasible point, the point is either removed by the cut and in **no** child, or kept and in **exactly one** — 121 pairs. A partition that silently lost efficient points would still agree with the optimum on lucky instances.

Bounds and the anytime gap. Boxes are settled best-first on a bound inherited from the parent, and a box whose bound cannot beat the incumbent is dropped whole without ever being split. The open list is therefore a certified upper bound on everything still unfound, so `time_budget` and the gap work exactly as they do for the paper's method.

And the margin grows with the number of criteria. Everything above has `p = 3`. Two things grow with `p` and pull against each other: the split produces `p` children, so the box list branches wider; but the non-dominated set grows too, so the decision-space method needs more cuts, and every cut costs it `p` binaries and `p+1` rows. Four instances at each `p`, `n = 6`:

p	decision space	criterion space	
2	0.61 s	0.35 s	1.74×
3	1.02 s	0.34 s	2.99×
4	4.21 s	0.61 s	6.92×
5	112.77 s	1.09 s	**103×**
6	26.15 s	1.08 s	24.3×

All twenty agreed and all twenty were proved optimal on both sides. The worst case for the paper's method was `p = 5` on **six variables**: 110.95 s against 0.64 s, a factor of 173, and within a minute of the two-minute budget the measurement allowed it.

The two forces are not comparable, which is the point. Branching multiplies a count of **cheap, fixed-size** models; the binaries multiply the **cost of one model**, and they accumulate. So more criteria hurt the decision-space method far more than they hurt this one — the opposite of what one might expect from "the split makes `p` children", and the strongest single sign that the difference here is structural rather than a matter of tuning.

Why both methods stay. The value of this package is that it is a reference implementation of the paper, and this search is not in the paper. It is offered beside it, not in place of it.

An exact–metaheuristic hybrid

`metaheuristic.optimize_hybrid_metaheuristic` runs a Pareto local search first and hands the box search an incumbent. Measured on 18 instances:

```
9.50s  ->  7.60s     1.25x     10 of 18 faster
```

Read by regime rather than by the total, because the heuristic's cost is nearly fixed:

	exact only	metaheuristic + exact	
`hardest n=10`	49 boxes, 3.02 s	34 boxes, **1.94 s**	**1.56×**
`n=8 s0`	16 boxes, 0.53 s	10 boxes, **0.34 s**	1.53×
`p=7 s1`	197 boxes, 1.36 s	120 boxes, **0.95 s**	1.43×
`p=5 s1`	6 boxes, **0.02 s**	6 boxes, 0.03 s	0.56×

It wins where the search is expensive and loses where there was nothing to win. The **ceiling**, measured by handing the search the true optimum for free, is 1.42×; at 1.25× the heuristic reaches 88 % of it.

Why it can pay here and cannot pay there. Handing the paper's method the true optimum for free at iteration 1 saves **zero iterations** on all three reference instances — what gets cut is decided by the efficiency test on `x_l`, not by `Phi_opt`. A box, by contrast, is discarded **whole** when its bound fails to beat the incumbent. So the worth of a heuristic incumbent depends on which exact method it is hybridised with: no mechanism in one, a real one in the other. That contrast is measured in both directions and is the more interesting of the two results.

Where correctness lives. The incumbent prunes whole boxes, so the value of a dominated point could prune away the true optimum. Nothing leaves the metaheuristic unverified: every candidate is confirmed efficient by the exact test of Theorem 1, or walked to an efficient point whose value is returned in its place. The heuristic decides where to look; it never decides what is true. A test pins exactly this — every incumbent efficient, and never above the optimum, against the independent scan.

Two things had to be right, and both were wrong first. The neighbourhood needs **swaps** (one coordinate down, another up), not only unit steps: from a maximal point a step up is infeasible and a step down lowers every criterion with non-negative coefficients, so the archive refuses it and the walk dies at once — three archive points, an incumbent at 13 % of the optimum. With swaps: 28–69 points and 60–100 %. And the archive **is** the search: restarting from fresh random points throws it away between walks, and four times the effort did not fix that monotonically (0.360, 0.280, 0.458).

What costs what. Not the walk — it is 0.01–0.08 s and barely sensitive to its budget. The cost is the verification, one integer program per candidate, which is the price of soundness. Examining the single best archive member gave exactly the same incumbent as examining four on every instance tried, at a quarter to a half of the cost, so `candidates` defaults to 1.

Performance. The method is MILP-bound: every iteration maximises `Phi` over a region carrying `p` extra binaries and `p+1` extra rows per cut already made, and that one sub-problem is 94 % of the run time. Profiling drove every choice below; on the `n = 4` instance they compound to **351 s → 0.41 s**.

Making each linear program cheaper. Reduced costs are **maintained** through the pivots rather than recomputed from `c_B' B^-1 A` — `O(n)` per pivot instead of `O(mn)`, which with exact rationals dominated everything. Pivot updates skip the zeros of the pivot row. A **crash basis** puts the ≤-slacks straight into the initial basis, so phase I is skipped whenever they already cover every row.

Making the sub-problems smaller. The Sylva–Crema rows are emitted in ≤ form so their slacks feed that crash basis, and the `y_i <= 1` rows of equation (5) are dropped — they are redundant,

since `y_i >= 2` only imposes a stronger requirement and leaves the union over the integer `y` unchanged. `Q(x~)` is solved over `D` rather than over the truncated region (the criterion slice is provably untouched by the cuts), which keeps every accumulated binary out of it. And when `V'x + beta > 0` holds on the whole relaxation, the sign split is skipped, halving every fractional sub-problem.

Searching less. The branch & bound takes an **objective cutoff**: the algorithm never needs the exact maximum over the truncated region, only whether it beats the incumbent, so handing the incumbent over prunes most of the tree in precisely the late iterations where the accumulated binaries would bite.

Not re-solving what the parent already solved. This was the big one. A child node differs from its parent by a single bound row, yet solving it from scratch paid a full phase I — measured at **20.6 phase-I pivots per node against 4.1 phase-II ones**, five sixths of the work thrown away. Children are now warm started: the branch row is appended to the parent's optimal tableau and a dual-simplex restoration repairs the one infeasible row, in **2.8 pivots on average**. A child whose restoration stalls falls back to a cold solve, so the warm start can cost time but never correctness — and a test forces every restoration to stall and checks that warm and cold agree on every instance. On a hard `n = 10` instance: 2996 children, zero stalls, 21 cold solves in total, and 657 children proved empty by the dual infeasibility certificate without solving anything at all.

Validation. `tests/` checks the paper's numbers step by step (`M = (−3,−3)`, `x_1 = (0,0)` with `Phi = 1`, `psi* = 2`, `X_opt = (3,3)`, `Phi_opt = 5/17`, `|D| = 11`, `|E(P_D)| = 7`), verifies that the three reference methods agree, and cross-checks 60 random bi-/tri-objective instances against exhaustive enumeration plus 12 larger ones (up to 404 feasible points) against the `Phi`-ordered scan. Eight `n = 10` instances with up to 34635 feasible points are verified against the scan in `examples/large_example.py`, and the certificate is checked to prove the real optimum while rejecting a merely efficient point, a merely good one and an infeasible one.

The augmented weighted Tchebychev program: a reference from the literature

`tchebychev.py` implements the scalarisation the surrounding literature uses to generate efficient points, and measures it against the two generators already in the package. It is here because the references this work sits next to — Chaabane, Brahmi and Ramdani (2012), Younsi-Abbaci and Moulai (2021) — build on it, so "how does the proposed hybrid compare with the existing methods" has to be answered against it and not only against the weighted sum.

With `z*` the ideal point, `z*_k = max { Z_k(x) : x in D }`, weights `w > 0` and an augmentation `rho > 0`:

```
min   max_k  w_k ( z*_k - Z_k(x) )  +  rho * sum_k ( z*_k - Z_k(x) )
s.t.  x in D
```

which linearises with one continuous variable `lambda >= w_k (z*_k - Z_k(x))` for each `k`. So each weight vector costs one integer program on a model with `n + 1` variables and `p` extra rows, plus `p` programs paid once for `z*`.

Why the optimum is efficient, for any `w > 0` and `rho > 0`. Let `x*` solve it and suppose some `y` dominated it. Every deviation `z*_k - Z_k(y)` is then no larger than at `x*`, so the `max` term does not increase, while the sum strictly decreases because one term strictly does. The objective at `y` would be strictly smaller — contradiction. Without `rho` the second half of that argument is unavailable and a merely **weakly** efficient point can minimise the program; that is the whole job of the augmentation, and a `rho <= 0` is refused rather than quietly accepted.

What it buys over a weighted sum. A weighted sum maximises only at supported efficient points, those on the convex hull of the criterion image. On the paper's instance three of the seven efficient points are unsupported: the Tchebychev program reaches all three, and no strictly positive weighted sum reaches any (`test_tchebychev_reaches_efficient_points_no_weighted_sum_can`).

Over the 20 random instances of `examples/tchebychev_study.py`, from the same grid of 27 weight vectors, it reached 112 of 175 efficient points against the weighted sum's 66, and 29 of the 54 unsupported ones against **0**; all 540 optima it returned were efficient. A wider sweep over 40 instances agrees: 216/369 against 135/369, 56 unsupported against 0, 1080 optima and no inefficient one.

What it does not buy: a better seed. `tchebychev_incumbent` is a drop-in alternative to `metaheuristic_incumbent` — both return an efficient point and its `Phi` for the box search to start from. Over the 18 instances of the hybrid study:

seed	total	seed cost	exactly optimal
none	9.64 s	—	—
Pareto local search	**7.97 s** (1.21×)	0.57 s	12 of 18
augmented Tchebychev	13.50 s (0.71×)	4.25 s	5 of 18

It is slower than doing nothing. Both halves of that are structural. The cost is `p + 1` integer programs on an enlarged model, and it is not the reference point's fault — `z*` took 0.01–0.13 s of the seed while the scalarisations took 0.13–0.85 s. The quality is the real finding: **the program never looks at `Phi`**. Its weights steer in criterion space, relative to `z*`; the local search is guided by `Phi` at every move. A good spread of efficient points is not a good `Phi`, and the literature's generator is optimising the wrong thing for this objective.

The ordering is stable across instance families; the size of the win is not. On the looser family in `examples/tchebychev_study.py` the Pareto seed is exactly optimal on 15 of 24 instances and still buys only 1.05×, because that search is not bound-limited. Seed **quality** and seed **value** are different quantities — the same lesson as the zero-iteration result for the paper's method. What does not move: Tchebychev loses to not seeding at all on every family tried (0.73×, 0.91×), and the Pareto seed ties or wins on every one.

As front generators the two trade places on a different axis. On 20 instances at `n = 4` the Tchebychev grid reached 64% of `E(P_D)` in 2.1 s and the Pareto archive 99% in 0.04 s. But the archive filters by dominance among the points it has seen, which is not the exact test of Definition 1: over a wider sweep one member in roughly 2000 was not efficient — reproducibly, a concrete case being a four-variable instance where the archive keeps `(2,1,1,1)` — while every Tchebychev optimum is efficient by construction. Cheap, near-complete, uncertified against expensive, partial, certified; and rare is not never, which is exactly why `metaheuristic_incumbent` puts the archive's best through `test_efficiency` (and `repair_to_efficient` when it fails) before any bound is allowed to cross into the exact search.

Linear criteria only. `z*_k - Z_k(x)` has to be linear in `x` for those rows to be constraints of an integer linear program. With a ratio it is not, and `ideal_point` and `augmented_tchebychev_efficient` both refuse rather than return something unfounded. The references are for linear criteria too.

A measurement note. An earlier harness scored a seed by the ratio `Phi(seed) / Phi*`. That ratio **inverts when `Phi*` is negative**, as it is on some of these instances, so a worse seed could score above 1. The tables here use the non-negative gap `(Phi* - Phi(seed)) / |Phi*|`, which is zero exactly when the seed is optimal and well defined whatever the sign.

What an explicit `E(P_D)` buys

The decision-space methods need **structure in `Phi`** and nothing of `E(P_D)`; this package's criterion-space route needs **`E(P_D)`** and nothing of `Phi`. A branch-and-cut for a linear fractional `Phi` derives its cuts and bounds from the ratio form; given a sum of two ratios it does not slow down, it stops applying. Over a proved-complete `E(P_D)` that case costs what one ratio costs:

```
from lfp_efficient import (complete_efficient_set,
                           maximize_over_efficient_set,
```

```
                                    nondominated_over_efficient_set)

result = complete_efficient_set(problem)
maximize_over_efficient_set(result, lambda x: phi1(x) + phi2(x))  # 47/35
nondominated_over_efficient_set(result, [phi1, phi2])             # 4 of 7
```

Sums, maxima, products, ratios of quadratics and black boxes are all exact; each is checked against brute force in the suite. Both entry points refuse a `CompleteSet` that is not proved complete, since a maximum over a truncated set is still a number and nothing downstream could tell the difference.

Measured: against running the exact method once per decision function, building `E(P_D)` first pays after a median of 13 functions (range 1–2117, so a direction and not a prediction). **For a single linear fractional `Phi` the decision-space method is often the better buy.** The break-even is 1 only outside the class it accepts.

How far the method reaches

The slice argument needs nothing of the criteria, so what limits the method is only whether a level set can be enumerated — and that is answerable exactly.

- **Limit.** One-hyperplane level sets with data affine in the level **iff** linear fractional. Within that shape the fractional case is maximal.
- **Reach.** The enumerator takes equations and inequalities, so it is enough that `{x : Z(x) = v}` be a **finite union of polyhedra**. That reaches `|ratio|` and `max`/`min` of ratios, which are not fractional.
- **The other half reaches less.** The front search needs both sides of its split to be half-spaces, so it requires criteria with half-space sublevel sets (a necessary condition; sufficiency is established for the fractional class, by construction) — essentially the fractional case. `max`-of-ratios has a polyhedral sublevel set whose complement is a union, and one split cannot express a union.

So there are two reaches, not one, and the smaller names the method: this is a MOILFP package because of its front, not its slice. A criterion between the two can have its efficient set completed exactly here, with its front supplied from elsewhere.

Modules

File	Contents
`rational.py`	exact arithmetic helpers
`simplex.py`	tableau simplex, linear and linear-fractional pricing, warm start
`milp.py`	warm-started branch & bound; sign-split fractional MILP
`model.py`	`Model`, `MOILFP` (and `MOILP` as its linear case), `FractionalObjective`, `e_row`
`efficiency.py`	exact efficiency test, dominance repair, dominance cut, `Q(x~)`
`edges.py`	reduced gradient, `Gamma_l`, `theta0`, edge walk
`algorithm.py`	the main loop, with a full iteration trace
`enumeration.py`	the three independent reference methods used by the tests
`criterion_space.py`	the box search in criterion space, and two optimisations that lost
`hybrid.py`	the exact-exact hybrid: the paper's method handing over to the box search
`metaheuristic.py`	Pareto local search with an archive, and the seeded hybrid
`tchebychev.py`	the augmented weighted Tchebychev program, measured as a reference

File	Contents
`front.py`	the complete non-dominated set, proved complete in criterion space
`complete.py`	`E(P_D)` itself: front slices, exact-rational variable elimination, the box guards
`subset.py`	a subset of `E(P_D)`, delivered as a subset and saying so
`generated.py`	the Pareto walk that seeds the enumeration, and the probe order that makes it pay
`over.py`	queries over an explicit `E(P_D)`: **any** computable decision function, or several at once
`levelsets.py`	how far the slice reaches: the characterisation, its two widenings, and branch builders for criteria outside the ratio class